

Studiu experimental asupra eficienței unor algoritmi de sortare

Vasile-Gabriel Ginga
vasile.ginga06@e-uvvt.ro

2026

Rezumat

Această lucrare prezintă un studiu experimental asupra principalilor algoritmi de sortare, analizând performanța acestora în funcție de timpul de execuție și consumul de memorie, pe diferite tipuri de date.

1 Introducere

Sortarea reprezintă una dintre cele mai importante operații în informatică, fiind utilizată în numeroase aplicații, de la baze de date până la procesarea informației. Deși complexitatea teoretică oferă o estimare generală a performanței, comportamentul real al algoritmilor poate varia semnificativ în funcție de natura datelor de intrare.

În această lucrare analizăm mai mulți algoritmi clasici de sortare, urmărind comportamentul lor pe seturi de date diferite, precum și impactul acestora asupra timpului de execuție și consumului de memorie.

2 Descrierea algoritmilor

Algoritmii analizați în această lucrare sunt:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- QuickSort
- HeapSort

Algoritmii Bubble Sort, Selection Sort și Insertion Sort au complexitate $O(n^2)$ și sunt considerați algoritmi simpli, utilizați în principal pentru scopuri educaționale sau pentru seturi mici de date [1].

În contrast, Merge Sort [2], QuickSort [3] și HeapSort [4] au complexitate $O(n \log n)$ și sunt utilizați în aplicații reale datorită eficienței lor [5].

3 Metodologie și implementare

Experimentele au fost realizate pe un sistem bazat pe procesor AMD Ryzen 7 5800X, cu 48GB RAM și placă video NVIDIA RTX 4060.

Seturile de date utilizate au fost:

- random
- sorted
- reverse
- almost sorted
- few unique

Pentru fiecare algoritm s-au măsurat:

- timpul de execuție (ms)
- memoria utilizată (KB)

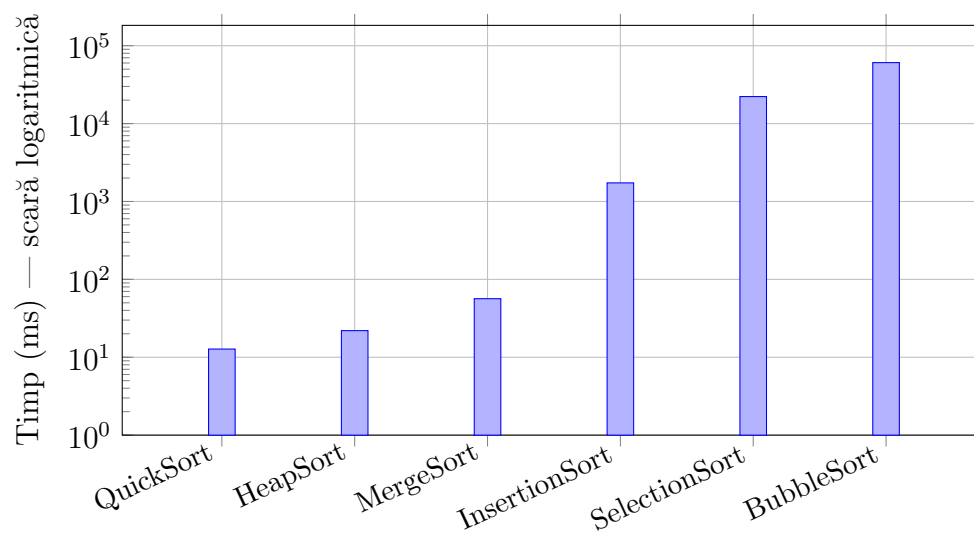
Graficele de timp din secțiunea următoare folosesc scară logaritmică pe axa Y. Aceasta este necesară deoarece diferențele dintre algoritmi acoperă mai multe ordine de mărime (de la < 1 ms până la $> 60\,000$ ms), iar o scară liniară ar face barele algoritmilor eficienți invizibile. Algoritmii sunt ordonați de la cel mai rapid la cel mai lent pentru fiecare tip de date, astfel încât forma graficului reflectă direct ierarhia performanței în acel scenariu.

Codul sursă complet al implementărilor, împreună cu seturile de date și scripturile de măsurare, este disponibil public la adresa:

<https://github.com/vasileginga/Eficienta-Algoritmi-de-sortare>

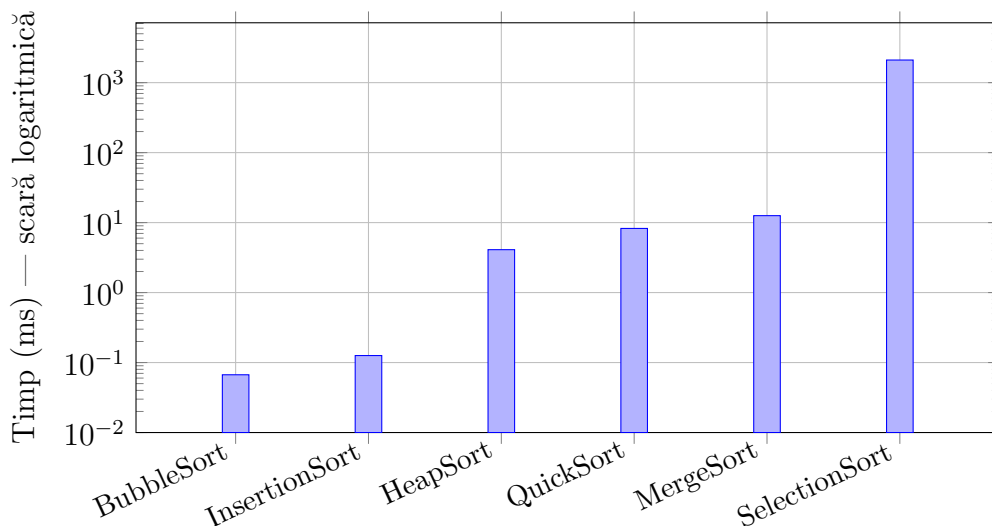
4 Analiză

4.1 Date random



Pentru date aleatoare, algoritmi din clasa $O(n \log n)$ domină clar. QuickSort este cel mai rapid (≈ 12.7 ms), urmat de HeapSort (≈ 22.0 ms). MergeSort înregistrează un timp mai ridicat decât QuickSort și HeapSort, ceea ce poate fi explicat prin costul alocărilor de memorie auxiliară în implementarea utilizată. Algoritmi $O(n^2)$ sunt complet impracticabili: InsertionSort ajunge la $\approx 1\,733$ ms, SelectionSort la $\approx 22\,234$ ms, iar BubbleSort la $\approx 60\,670$ ms — de aproximativ 5000 de ori mai lent decât QuickSort.

4.2 Date sortate

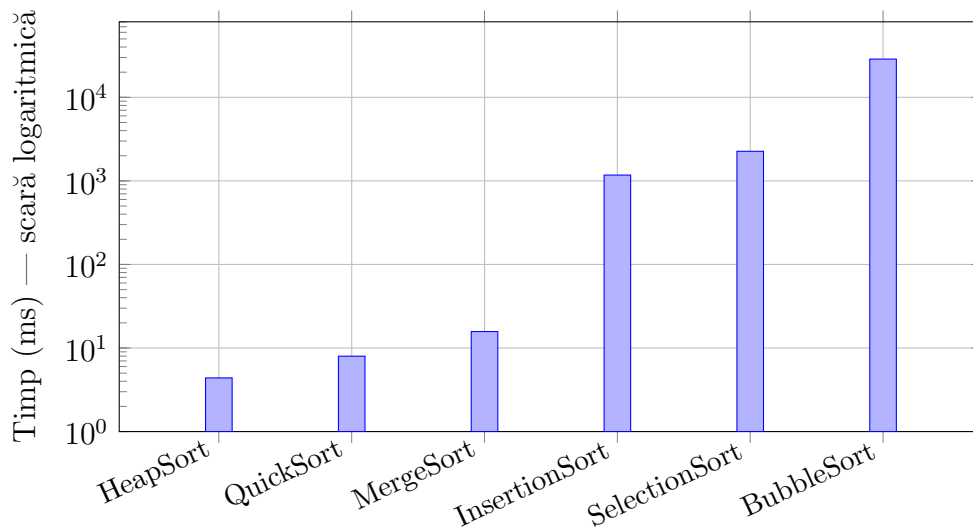


În cazul datelor deja sortate, se observă o schimbare majoră a comportamentului algoritmilor. BubbleSort și InsertionSort devin extrem de rapide, având timpi aproape neglijabili. Acest lucru se explică prin faptul că ambii algoritmi pot detecta rapid că lista este deja ordonată și nu mai efectuează operații inutile.

InsertionSort beneficiază de caracterul său adaptiv, iar BubbleSort, în implementarea utilizată, oprește execuția imediat ce nu mai sunt necesare interschimbări.

În schimb, algoritmi din clasa $O(n \log n)$ (QuickSort, MergeSort, HeapSort) nu exploatează această proprietate și mențin timpi relativ constanți.

4.3 Date inverse

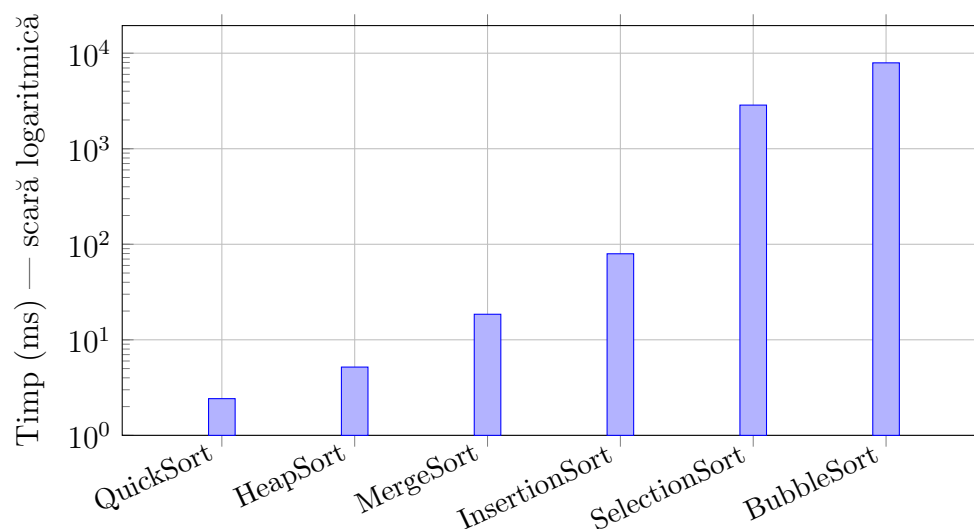


Datele invers sortate reprezintă un caz nefavorabil pentru algoritmi simpli. BubbleSort și InsertionSort înregistrează timpi foarte mari, deoarece fiecare element trebuie mutat în poziția corectă.

SelectionSort rămâne relativ constant, deoarece nu depinde de ordinea inițială a datelor.

În cadrul algoritmilor eficienți, se observă că HeapSort devine cel mai rapid, depășind QuickSort. Acest lucru apare deoarece QuickSort poate fi influențat de alegerea pivotului, în timp ce HeapSort menține o performanță stabilă indiferent de distribuția datelor.

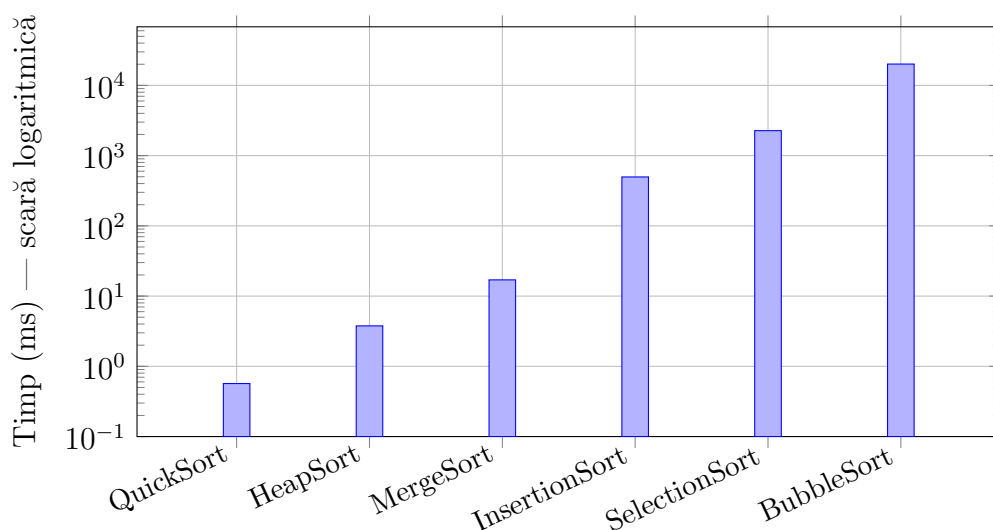
4.4 Date aproape sortate



Datele aproape sortate reprezintă un caz intermediar, în care algoritmi adaptivi beneficiază parțial de ordinea preexistentă. InsertionSort înregistrează un timp de ≈ 79.6 ms, semnificativ mai mic decât pe date aleatoare (≈ 1733 ms), demonstrând că adaptivitatea sa este activă chiar și atunci când datele nu sunt complet ordonate. BubbleSort, deși și el adaptiv, ajunge totuși la ≈ 7926 ms, deoarece numărul de interschimbări rămâne ridicat când elementele sunt deplasate față de poziția corectă.

SelectionSort (≈ 2862 ms) rămâne neschimbat față de celelalte scenarii, confirmând lipsa sa de adaptivitate. Algoritmii $O(n \log n)$ domină în continuare: QuickSort (≈ 2.4 ms) este cel mai rapid, urmat de HeapSort (≈ 5.2 ms) și MergeSort (≈ 18.5 ms). Față de datele aleatoare, QuickSort este chiar mai rapid în acest scenariu, ceea ce sugerează că structura parțial ordonată favorabilă alegerii pivotului reduce numărul de partiționări necesare.

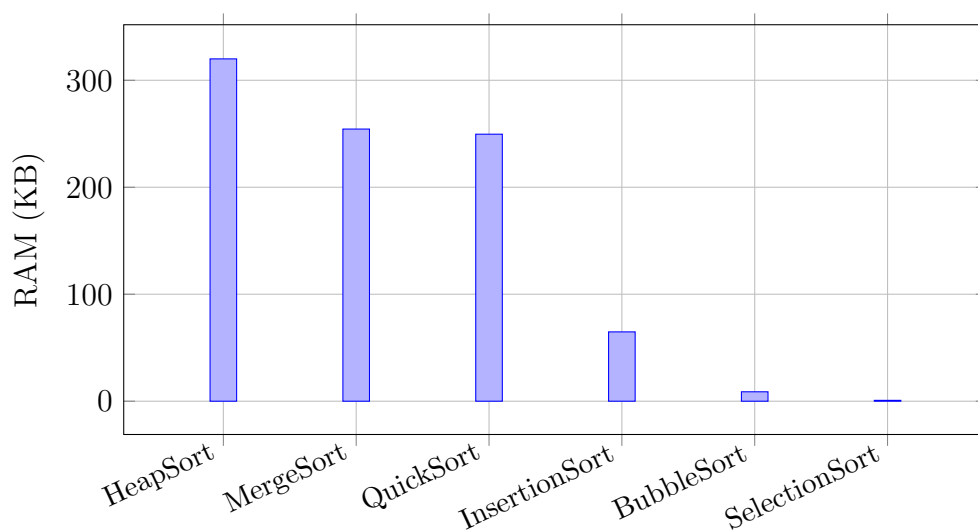
4.5 Date cu puține valori unice



Setul de date cu puține valori unice reprezintă un caz special în care multe elemente au aceeași valoare. QuickSort înregistrează cel mai bun timp din toate scenariile testate (≈ 0.57 ms), ceea ce indică faptul că implementarea utilizată gestionează eficient elementele egale, probabil prin partitionare în trei căi sau o variantă similară. HeapSort (≈ 3.8 ms) și MergeSort (≈ 17.0 ms) își mențin performanța similară cu celelalte scenarii, nefiind influențați semnificativ de numărul de valori distincte.

InsertionSort (≈ 497 ms) se comportă mai bine decât pe date aleatoare sau inverse, dar mult mai slab decât pe date sortate, deoarece elementele egale nu elimină necesitatea comparațiilor. BubbleSort (≈ 20129 ms) și SelectionSort (≈ 2267 ms) rămân impractici, confirmând că apartenența la clasa $O(n^2)$ este determinantă indiferent de distribuția valorilor.

4.6 Consum de memorie



În graficul de memorie se observă diferențe clare între algoritmi.

HeapSort, MergeSort și QuickSort folosesc mai multă memorie, deoarece implică operații suplimentare precum recursivitate sau structuri auxiliare. Chiar dacă teoretic

QuickSort este considerat eficient din punct de vedere al memoriei, în implementarea testată acesta utilizează stiva de apeluri și execuție în thread-uri, ceea ce duce la un consum mai ridicat.

MergeSort utilizează memorie suplimentară pentru vectorii auxiliari, iar HeapSort implică operații suplimentare pentru gestionarea heap-ului.

În schimb, algoritmi simpli (BubbleSort, SelectionSort, InsertionSort) sunt algoritmi in-place, ceea ce înseamnă că lucrează direct pe vector fără a alocă memorie suplimentară semnificativă. Din acest motiv, aceștia au cel mai mic consum de memorie în grafic.

5 Comparații cu literatura de specialitate

Lucrarea *Performance Comparison of Different Sorting Algorithms* de Prajapati et al. (IJLTEMAS, 2017) [6] este utilizată ca referință pentru compararea prezentului studiu pe trei axe: problema abordată, contextul cercetării și metoda utilizată. Pe fiecare axă sunt evidențiate atât similitudinile, cât și diferențele față de lucrarea de referință, împreună cu avantajele și dezavantajele care decurg din acestea.

5.1 Problemă

Ambele lucrări abordează aceeași problemă fundamentală: compararea performanței algoritmilor clasici de sortare. Prajapati et al. analizează un spectru mai larg, incluzând 15 algoritmi (BubbleSort, Modified BubbleSort, SelectionSort, Enhanced SelectionSort, InsertionSort, MergeSort, QuickSort, variante paralele de QuickSort, MQ Sort, RadixSort, CountingSort, HeapSort, BucketSort), cu accent pe clasificarea lor după complexitate teoretică, stabilitate și domenii de aplicabilitate. Prezentul studiu se concentrează pe 6 algoritmi reprezentativi (BubbleSort, SelectionSort, InsertionSort, MergeSort, QuickSort, HeapSort), cu accent pe comportamentul lor practic măsurat pe date reale.

Avantaj: Focalizarea pe un set restrâns de algoritmi permite o analiză mai profundă a comportamentului fiecăruia pe tipuri variate de date, evidențiind nuanțe care nu reies dintr-un survey teoretic larg.

Dezavantaj: Absența algoritmilor non-comparativi (RadixSort, CountingSort) și a variantelor paralele limitează relevanța concluziilor pentru aplicații care procesează volume mari de date sau rulează pe arhitecturi multi-core.

5.2 Context

Prajapati et al. operează exclusiv în context teoretic: complexitățile raportate (best/average/worst case) sunt cele matematice standard, independent de orice hardware sau implementare specifică. Lucrarea nu descrie un mediu de testare, nu rulează cod și nu produce timpi mășurați — concluziile sunt valabile universal, dar nu reflectă comportamentul real pe un sistem concret.

Prezentul studiu operează într-un context experimental precis: hardware specific (AMD Ryzen 7 5800X, 48 GB RAM), implementări concrete în cod, seturi de date cu $n = 100\,000$ elemente și cinci tipuri de distribuție (random, sorted, reverse, almost sorted, few unique). Rezultatele sunt prin urmare dependente de context, dar reflectă comportamentul real al implementărilor testate.

Avantaj: Contextul experimental permite identificarea unor discrepante față de teoria clasică care ar fi invizibile într-un studiu pur teoretic — de exemplu, HeapSort con-

sumând mai multă memorie decât MergeSort datorită utilizării thread-urilor, sau MergeSort înregistrând timpi neașteptat de mari pe date aleatoare din cauza overhead-ului de alocare.

Dezavantaj: Rezultatele nu sunt generalizabile direct la alte hardware-uri sau implementări. Un alt sistem sau o altă implementare a acelorași algoritmi poate produce ierarhii diferite, spre deosebire de concluziile teoretice ale Prajapati et al. care rămân valabile indiferent de platformă.

5.3 Metodă

Prajapati et al. utilizează o metodă de tip *survey teoretic*: compilează și sintetizează informații din literatura existentă, prezintă tabele de complexitate și avantaje/dezavantaje pentru fiecare algoritm, și identifică domenii de aplicabilitate. Nu există experimente proprii, cod rulat sau date măsurate.

Prezentul studiu utilizează o metodă *experimentală cantitativă*: implementează algoritmi, îi rulează pe seturi de date controlate, măsoară timpii de execuție în milisecunde și consumul de RAM în kilobytes, și vizualizează rezultatele prin grafice cu scară logaritmică pentru a face vizibile diferențele dintre toate clasele de algoritmi simultan.

Avantaj: Metoda experimentală produce date concrete și verificabile, permite comparații directe pe aceleași seturi de date și evidențiază comportamente care nu reies din analiza teoretică (adaptivitatea BubbleSort pe date sortate, stabilitatea HeapSort pe date inverse etc.). Graficele cu scară logaritmică reprezintă o contribuție vizuală care lipsește complet din lucrarea de referință.

Dezavantaj: Metoda experimentală nu acoperă analiza domeniilor de aplicabilitate și nu discută proprietăți calitative precum stabilitatea sau comportamentul pe structuri de date non-array (liste înlănțuite, arbori), aspecte tratate explicit de Prajapati et al. De asemenea, reproducibilitatea rezultatelor depinde de accesul la același hardware și la aceleași implementări.

6 Concluzii

În urma analizei experimentale și a comparației cu literatura de specialitate, se poate concluziona că alegerea algoritmului de sortare trebuie realizată în funcție de natura datelor și de constrângerile aplicației. Rezultatele confirmă în linii mari predicțiile teoretice prezentate de Prajapati et al. [6], cu câteva excepții notabile generate de detaliile de implementare. QuickSort rămâne cea mai bună alegere în majoritatea cazurilor practice. HeapSort reprezintă o alternativă solidă când datele sunt invers sortate sau când consistența timpului de execuție este prioritară. Algoritmii simpli sunt limitați la scenarii specifice, dar rămân valoroși pe date deja sortate datorită proprietății lor adaptive. Discrepanțele față de teoria clasică, în special comportamentul MergeSort și consumul de memorie al HeapSort, evidențiază importanța evaluării experimentale ca complement necesar al analizei teoretice.

Bibliografie

- [1] Thomas H. Cormen et al., *Introduction to Algorithms*, 3rd, MIT Press, 2009, ISBN: 978-0-262-03384-8, URL: <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>.
- [2] John von Neumann, “First Draft of a Report on the EDVAC”, în *IEEE Annals of the History of Computing* 15.4 (1993), Reprodus din manuscrisul original din 1945, pp. 27–75, DOI: [10.1109/85.238389](https://doi.org/10.1109/85.238389), URL: <https://doi.org/10.1109/85.238389>.
- [3] C. A. R. Hoare, “Quicksort”, în *The Computer Journal* 5.1 (1962), pp. 10–16, DOI: [10.1093/comjnl/5.1.10](https://doi.org/10.1093/comjnl/5.1.10), URL: <https://doi.org/10.1093/comjnl/5.1.10>.
- [4] J. W. J. Williams, “Algorithm 232: Heapsort”, în *Communications of the ACM* 7.6 (1964), pp. 347–348, DOI: [10.1145/512274.512284](https://doi.org/10.1145/512274.512284), URL: <https://doi.org/10.1145/512274.512284>.
- [5] Donald E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd, Addison-Wesley, 1998, ISBN: 978-0-201-89685-5, URL: <https://www.informit.com/store/9780201896855>.
- [6] Purvi Prajapati, Nikita Bhatt și Nirav Bhatt, “Performance Comparison of Different Sorting Algorithms”, în *International Journal of Latest Technology in Engineering, Management & Applied Science* VI.VI (Iun. 2017), pp. 39–41, ISSN: 2278-2540, URL: <https://www.ijltemas.in/DigitalLibrary/Vol.6Issue6/39-41.pdf>.